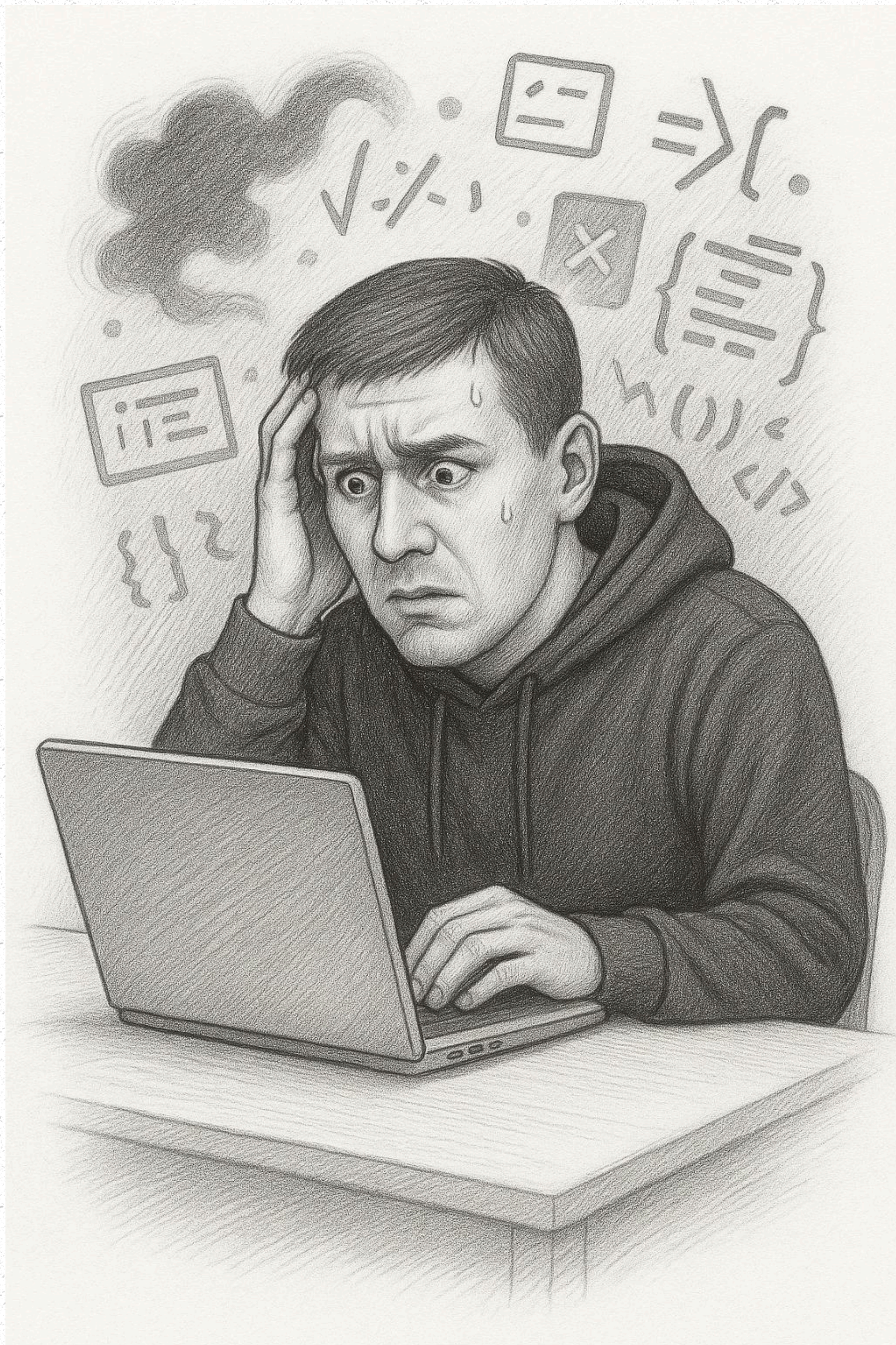


Dependency Injection Anti-Patterns

6 Mistakes Killing your ASP.NET Core Apps



01

Injecting Scoped and Transient Services Into Singletons

✗ Memory leaks that grow until your app crashes.

```
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddSingleton<UserService>();

public interface IUserRepository { }

public class UserRepository () : IUserRepository { }

public class UserService(IUserRepository repository)
{
    public void Create(User user)
    {
        return repository.Create(user);
    }
}
```



02

Injecting Scoped and Transient Services Into Singletons

✓ Fix: Start with transient by default. Move to transient only when you always need a fresh instance. Prefer scoped or singleton.

```
public class UserService(IServiceScopeFactory scopeFactory)
{
    public void Create(User user)
    {
        using var scope = scopeFactory.CreateScope();

        var repository = scope.ServiceProvider
            .GetRequiredService<IUserRepository>();

        repository.Create(user);
    }
}
```



03 Using Transient Services Everywhere

✗ Each transient service adds memory overhead per request.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddTransient<UserRepository>();
builder.Services.AddTransient<UserService>();
builder.Services.AddTransient<EmailSender>();
builder.Services.AddTransient<ReportGenerator>();

var app = builder.Build();
app.Run();
```



04 Using Transient Services Everywhere

✓ Fix: Start with transient by default. Move to transient only when you always need a fresh instance. Prefer scoped or singleton.

```
var builder = WebApplication.CreateBuilder(args);

// Lightweight
builder.Services.AddTransient<IUserRepository>();

// Stateless
builder.Services.AddTransient<UserService>();

// Heavy, reusable
builder.Services.AddSingleton<EmailSender>();

// Needs per-request state
builder.Services.AddScoped<ReportGenerator>();

var app = builder.Build();
app.Run();
```



05

Thread-Unsafe Singletons Singleton

✗ Without proper synchronization, you'll face random crashes and data corruption.

```
// Custom not thread-safe Singleton
public class CounterService
{
    private static CounterService? _instance;
    private int _count;

    public static CounterService Instance =>
        _instance ??= new CounterService();

    public int Increment()
    {
        return ++_count; // ✗ Not thread-safe
    }
}
```



06

Thread-Unsafe Singleton Singleton

✓ Fix: Make singleton services immutable or implement proper locking.

```
var builder = WebApplication.CreateBuilder(args);  
builder.Services.AddSingleton<CounterService>();  
  
public class CounterService  
{  
    private int _count;  
  
    public int Increment()  
    {  
        lock (_lock)  
        {  
            return ++_count; // ✓ Thread-safe  
        }  
    }  
}
```



07 Service Locator Anti-Pattern

✗ This hides dependencies behind `IServiceProvider.GetService<T>()` calls scattered throughout your code.

```
public class UserService
{
    private readonly IServiceProvider _provider;

    public UserService(IServiceProvider provider)
    {
        _provider = provider;
    }

    public void Create(User user)
    {
        // ✗ hidden dependency
        var repo = _provider.GetService<IUserRepository>();
        repo?.Create(user);
    }
}
```



08 Service Locator Anti-Pattern

✓ Fix: Use constructor injection to make dependencies explicit.

```
public class UserService
{
    private readonly IUserRepository _repository;

    public UserService(IUserRepository repository)
    {
        _repository = repository;
    }

    public void Create(User user)
    {
        _repository.Create(user);
    }
}
```



09 Ignoring Disposal of Services

✗ Memory leaks that grow until your app crashes.

✓ Fix: Use using statements or `IServiceScope` when manually resolving services.

```
app.MapGet("/users", (IServiceProvider provider) =>
{
    // ✗ Not disposed
    var db = provider.GetRequiredService<UserDbContext>();
    return db.Users.ToList();
});

app.MapGet("/users", (IServiceProvider provider) =>
{
    // ✓ Scope handles disposal
    using var scope = provider.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    return db.Users.ToList();
});
```



Repost & Follow [Anton](#) for more content like this

Want to improve
.NET Skills?

Subscribe to my
[AntonDevTips](#)
newsletter

